Basic 2D Particle Filter (Monte Carlo Localization) Node

Introduction to Robotics: Perception and Planning (IRPP)

Inesh Roy
2024112015
Satvik Shrivastava
2024102029
Navanshu Gupta
2024102072

ECX Hell Yeah

Contents

1 Introduction **3**
1.1 Problem Statement . . . 3

2 Theoretical & Mathematical Background **3**
2.1 Bayesian Filtering . . . 3
2.2 Particle Filter – Sample-Based Belief . . . 4
2.3 Motion Model (Predict Step) . . . 4
2.4 Measurement Model (Update Step) . . . 5
2.5 Resampling . . . 6
2.5.1 Low-Variance (Systematic) Resampling . . . 6
2.5.2 Particle Deprivation and Recovery . . . 7
2.6 Complete MCL Algorithm . . . 7
2.7 Convergence and Particle Count . . . 7

3 Implementation **8**
3.1 System Overview . . . 8
3.2 Test Maps . . . 8

4 Results **9**
4.1 RViz Visualization . . . 9
4.2 Localization Error Over Time . . . 10
4.3 Effect of the Number of Particles . . . 11

5 Comparison with Existing MCL Package **12**
5.1 Overview of `nav2_amcl` . . . 12
5.2 Feature Comparison . . . 13
5.3 Experimental Setup . . . 13
5.4 Results . . . 13
5.5 Effect of Map Symmetry . . . 14

6 Conclusion **15**
6.1 Summary . . . 15
6.2 Key Findings . . . 15

1 Introduction

Localization—determining a robot's pose (position and orientation) within a known environment—is one of the most fundamental problems in mobile robotics. Without an accurate estimate of its own pose, a robot cannot plan collision-free paths, manipulate objects, or meaningfully interact with its surroundings.

Monte Carlo Localization (MCL), also known as the **Particle Filter** approach to localization, overcomes these limitations by representing the robot's belief as a set of weighted samples (*particles*) drawn from the state space. Each particle is a hypothesis about the robot's true pose. By recursively applying a *motion model* (predict step), a *measurement model* (update step), and a *resampling* step, the particle cloud converges from a uniform spread across the map to a tight cluster around the true robot pose.

1.1 Problem Statement

This involves building a particle filter to localize a robot moving in a 2D grid world with known landmarks. The robot does not know its starting position, so it scatters particles (hypotheses) across the map. As the robot moves (odometry) and senses landmarks (range/bearing sensors), particles that do not match the sensor readings are removed, and new particles are generated around the best-fitting positions. The simulation shows the particles converging on the actual robot position over time.

- **Rviz Visualization:** A visualization showing the robot's path, landmarks, and particles (colored dots) converging towards the actual robot position.
- **Error Reduction Plot:** A plot illustrating the average error reduction over time.
- **Effect of Particle Number:** An explanation of how the number of particles affects the accuracy of the localization process.
- **Comparison to Existing MCL Package:** A comparison between the custom implementation and an existing Monte Carlo Localization (MCL) package.

2 Theoretical & Mathematical Background

2.1 Bayesian Filtering

The particle filter is a *recursive Bayesian filter*. Let the robot's state at time t be denoted $\mathbf{x}_t = (x_t,\, y_t,\, \theta_t)^\top$, comprising its 2-D position (x, y) and heading θ.

Belief. The **belief** $\mathrm{bel}(\mathbf{x}_t)$ is the posterior probability density over the robot's state given all controls $\mathbf{u}_{1:t}$ and observations $\mathbf{z}_{1:t}$:

$$\mathrm{bel}(\mathbf{x}_t) = p(\mathbf{x}_t \mid \mathbf{z}_{1:t},\, \mathbf{u}_{1:t}). \tag{1}$$

Prediction (Chapman–Kolmogorov). Incorporating the latest control $\mathbf{u}_t$ yields the *predicted belief* (also called the *prior*):

$$\overline{\mathrm{bel}}(\mathbf{x}_t) = \int p(\mathbf{x}_t \mid \mathbf{u}_t,\, \mathbf{x}_{t-1})\, \mathrm{bel}(\mathbf{x}_{t-1})\, d\mathbf{x}_{t-1}. \tag{2}$$

Here,

- $p(\mathbf{x}_t \mid \mathbf{u}_t, \mathbf{x}_{t-1})$ is the **motion model** (state transition probability).
- The integral marginalizes over all possible previous states.

Update (Bayes' Rule). Upon receiving an observation $\mathbf{z}_t$, the belief is corrected:

$$\text{bel}(\mathbf{x}_t) = \eta \, p(\mathbf{z}_t \mid \mathbf{x}_t) \, \overline{\text{bel}}(\mathbf{x}_t), \tag{3}$$

where

- $p(\mathbf{z}_t \mid \mathbf{x}_t)$ is the **measurement model** (likelihood of the observation given the state).
- η is a normalization constant ensuring the belief integrates to 1.

The Markov Assumption. Equations (2) and (3) rely on the **Markov assumption**: conditioned on $\mathbf{x}_{t-1}$, the state $\mathbf{x}_t$ is independent of all earlier states and observations.

2.2 Particle Filter – Sample-Based Belief

The **particle filter** sidesteps this by represents the belief with a finite set of N weighted samples:

$$\mathcal{X}_t = \left\{ (\mathbf{x}_t^{[i]}, \; w_t^{[i]}) \right\}_{i=1}^{N}, \quad \sum_{i=1}^{N} w_t^{[i]} = 1. \tag{4}$$

Each particle $\mathbf{x}_t^{[i]} = (x^{[i]}, y^{[i]}, \theta^{[i]})^\top$ is a hypothesis about the robot's pose, and $w_t^{[i]}$ is its importance weight reflecting how well that hypothesis explains the observations.

The empirical density approximation is:

$$\text{bel}(\mathbf{x}_t) \approx \sum_{i=1}^{N} w_t^{[i]} \, \delta(\mathbf{x}_t - \mathbf{x}_t^{[i]}), \tag{5}$$

where $\delta(\cdot)$ is the Dirac delta function. As $N \to \infty$, the approximation converges to the true posterior.

2.3 Motion Model (Predict Step)

We use the **odometry motion model** (Thrun et al., *Probabilistic Robotics*). At each step the filter receives the robot's odometry pose, and the motion since the previous pose, $(\Delta x, \Delta y, \Delta\theta)$, is decomposed into an initial rotation, a translation and a final rotation:

$$\delta_{\text{trans}} = \sqrt{\Delta x^2 + \Delta y^2}, \tag{6}$$

$$\delta_{\text{rot1}} = \text{atan2}(\Delta y, \, \Delta x) - \theta_{t-1}, \tag{7}$$

$$\delta_{\text{rot2}} = \Delta\theta - \delta_{\text{rot1}}. \tag{8}$$

If the robot only rotates ($\delta_{\text{trans}} \approx 0$), δ_{rot1} is set to zero and the whole rotation goes into δ_{rot2}. All angles are wrapped into $(-\pi, \pi]$.

Adding Noise. Real odometry is corrupted by slip, wheel encoder errors, and unmodelled terrain effects. For every particle we add Gaussian noise to each component, with a standard deviation that grows with the size of the motion:

$$\hat{\delta}_{\text{rot1}} = \delta_{\text{rot1}} + \mathcal{N}\Big(0,\ \alpha_{\text{rot}}|\delta_{\text{rot1}}| + \alpha_{\text{trans}}\,\delta_{\text{trans}}\Big), \tag{9}$$

$$\hat{\delta}_{\text{trans}} = \delta_{\text{trans}} + \mathcal{N}\Big(0,\ \alpha_{\text{trans}}\,\delta_{\text{trans}} + \alpha_{\text{rot}}|\Delta\theta|\Big), \tag{10}$$

$$\hat{\delta}_{\text{rot2}} = \delta_{\text{rot2}} + \mathcal{N}\Big(0,\ \alpha_{\text{rot}}|\delta_{\text{rot2}}| + \alpha_{\text{trans}}\,\delta_{\text{trans}}\Big), \tag{11}$$

with $\alpha_{\text{rot}} = 0.05$ and $\alpha_{\text{trans}} = 0.02$. A small constant ($10^{-4}$) is added to each standard deviation so that the particles keep spreading slightly even when the robot is still.

Predict Step (per particle). For every particle i:

1. Decompose the observed motion into $\delta_{\text{rot1}}, \delta_{\text{trans}}, \delta_{\text{rot2}}$ (Eqs. (6)–(8)).
2. Add Gaussian noise to each component (Eqs. (9)–(11)).
3. Propagate the particle:

$$\theta' = \theta^{[i]} + \hat{\delta}_{\text{rot1}}, \tag{12}$$

$$x^{[i]} \leftarrow x^{[i]} + \hat{\delta}_{\text{trans}}\cos\theta', \qquad y^{[i]} \leftarrow y^{[i]} + \hat{\delta}_{\text{trans}}\sin\theta', \tag{13}$$

$$\theta^{[i]} \leftarrow \theta' + \hat{\delta}_{\text{rot2}}. \tag{14}$$

4. If the particle has moved into an obstacle or off the map, it is replaced by a random pose in free space.

This is equivalent to sampling from $p(\mathbf{x}_t \mid \mathbf{u}_t, \mathbf{x}_{t-1}^{[i]})$, thereby implementing Eq. (2) via Monte Carlo sampling.

2.4 Measurement Model (Update Step)

The robot is equipped with a **range-bearing sensor** that can detect landmarks within a maximum range $r_{\max}$. For a landmark l_j located at $(l_{j,x},\ l_{j,y})$ and a particle at $(x^{[i]}, y^{[i]}, \theta^{[i]})$, the expected measurement is:

$$\hat{r}_j^{[i]} = \sqrt{(l_{j,x} - x^{[i]})^2 + (l_{j,y} - y^{[i]})^2}, \tag{15}$$

$$\hat{\phi}_j^{[i]} = \operatorname{atan2}\Big(l_{j,y} - y^{[i]},\ l_{j,x} - x^{[i]}\Big) - \theta^{[i]}. \tag{16}$$

The actual sensor returns a noisy measurement $\mathbf{z}_j = (r_j, \phi_j)^\top$. We model the noise as independent Gaussians:

$$p(\mathbf{z}_j \mid \mathbf{x}^{[i]}, l_j) = \frac{1}{2\pi\,\sigma_r\,\sigma_\phi}\ \exp\left(-\frac{(r_j - \hat{r}_j^{[i]})^2}{2\sigma_r^2} - \frac{(\phi_j - \hat{\phi}_j^{[i]})^2}{2\sigma_\phi^2}\right), \tag{17}$$

where σ_r and σ_ϕ are the standard deviations of range and bearing noise respectively. The simulated sensor has a range of $r_{\max} = 8$ m and noise of 0.2 m in range and 0.05 rad in bearing. The filter uses slightly larger values, $\sigma_r = 0.3$ m and $\sigma_\phi = 0.1$ rad, so that it is not over-confident. Each reading carries the index of the landmark it belongs to, so the data association between measurements and landmarks is known.

Importance Weight. If K landmarks are observed at time t, the weight of particle i is updated as:

$$w_t^{[i]} = w_{t-1}^{[i]} \prod_{j=1}^{K} p(\mathbf{z}_j \mid \mathbf{x}_t^{[i]}, l_j). \tag{18}$$

After computing all weights, they are normalized:

$$\tilde{w}_t^{[i]} = \frac{w_t^{[i]}}{\sum_{k=1}^{N} w_t^{[k]}}. \tag{19}$$

The normalization constant $\eta = 1/\sum_k w_t^{[k]}$ corresponds to the η in Eq. (3).

Handling Bearing Wrap-Around. When computing the bearing difference $\phi_j - \hat{\phi}_j^{[i]}$, the result must be wrapped into $(-\pi, \pi]$ to avoid discontinuities:

$$\Delta\phi = \text{atan2}\Big(\sin(\phi_j - \hat{\phi}_j^{[i]}),\ \cos(\phi_j - \hat{\phi}_j^{[i]})\Big). \tag{20}$$

2.5 Resampling

After several update steps, the weight distribution typically becomes skewed: a few particles carry most of the probability mass while the rest have negligible weights. This *particle degeneracy* problem is addressed by **resampling**: drawing a new set of particles from the current set with replacement, where the probability of selecting particle i is proportional to $\tilde{w}_t^{[i]}$. In our implementation, resampling is performed **at every update step** to keep the particle cloud responsive.

2.5.1 Low-Variance (Systematic) Resampling

We employ the **low-variance resampling** algorithm, which is both efficient ($O(N)$) and introduces less sampling variance than multinomial resampling.

Algorithm 1 Low-Variance Resampling

Require: Particle set $\mathcal{X} = \{(\mathbf{x}^{[i]}, \tilde{w}^{[i]})\}_{i=1}^{N}$
Ensure: New particle set $\mathcal{X}'$ with uniform weights

```
r ~ U(0, 1/N)                                   ▷ single random draw
c ← w̃[1]
j ← 1
for i = 1 to N do
    U ← r + (i − 1)/N
    while U > c do
        j ← j + 1
        c ← c + w̃[j]
    end while
    Add (x[j], 1/N) to X'
end for
return X'
```

The key insight is that a single random number r generates N evenly spaced thresholds $\{r, r + 1/N, r + 2/N, \ldots\}$ that "sweep" across the cumulative weight distribution. This

ensures that particles with high weights are selected proportionally without the high variance of independent draws.

2.5.2 Particle Deprivation and Recovery

After many resampling steps, particles may lose diversity and collapse to a single mode. This is the *particle deprivation* problem. Our implementation uses two mitigations:

1. **Random particle injection:** At every resampling step, 15% of the particle budget is replaced with particles drawn uniformly from free space, ensuring continuous exploration.
2. **Lost detection:** If the largest unnormalized particle weight stays below $w_{\min} = 10^{-50}$ for 8 consecutive updates, the filter declares the robot lost and reinitialises all particles globally.

2.6 Complete MCL Algorithm

The following summarises the full MCL loop.

Algorithm 2 Monte Carlo Localization (MCL)

Require: Occupancy grid map $\mathcal{M}$ (for free space), landmark positions $\{l_j\}$, particle count $N = 500$

Initialization: Sample $\mathcal{X}_0 = \{\mathbf{x}_0^{[i]}\}_{i=1}^{N}$ uniformly over free space; set $w_0^{[i]} = 1/N$
for each time step $t = 1, 2, \ldots$ **do**
  Receive odometry displacement $(\Delta x, \Delta y, \Delta\theta)$ and landmark readings $\{\mathbf{z}_j\}$
  for $i = 1$ **to** N **do** ▷ PREDICT
    Decompose displacement and add noise (Eqs. (6)–(11))
    Propagate particle (Eqs. (12)–(14))
  end for
  for $i = 1$ **to** N **do** ▷ UPDATE
    Compute the expected range and bearing to each observed landmark (Eqs. (15)–(16))
    Compute $w_t^{[i]}$ from the measurement likelihood (Eqs. (17)–(18))
  end for
  Normalize weights: $\tilde{w}_t^{[i]} \leftarrow w_t^{[i]} / \sum_k w_t^{[k]}$
  $\mathcal{X}_t \leftarrow$ LOWVARIANCERESAMPLE($\mathcal{X}_t$) with 15% random injection ▷ RESAMPLE
  if $\max_i \tilde{w}_t^{[i]} < w_{\min}$ for T consecutive steps **then** ▷ LOST DETECTION
    Reinitialise all particles globally
  end if
end for

2.7 Convergence and Particle Count

The accuracy of a particle filter depends on the number of particles N: Increasing N improves the approximation of the true posterior distribution, reducing localization error. The computational cost of each iteration grows linearly with N. Too few particles may cause filter divergence, while too many particles reduce real-time performance. In our implementation, we use a fixed particle count of $N = 500$; Section 4.3 shows how the filter behaves with fewer and more particles.

3 Implementation

This section describes the software architecture and the from-scratch implementation of each MCL step as a ROS 2 node written in Python. Each node runs in its own Docker container (ROS 2 Humble), started together with Docker Compose.

3.1 System Overview

The implementation consists of the following components:

1. **Map server** (`nav2_map_server`) – publishes the occupancy grid of the 20 m × 20 m world (0.05 m per cell).
2. **Simulated robot** – drives around the map on its own while avoiding walls, publishes its ground-truth pose, and simulates the range-bearing sensor for 5 landmarks. Every 30 s it is teleported to a random free pose (the kidnapped robot problem).
3. **Custom MCL node** – subscribes to odometry and landmark readings, runs the particle filter, and publishes the estimated pose, its confidence (the largest particle weight) and the particle cloud.
4. **Scan bridge** – turns the robot's motion into odometry for both filters, and produces what `nav2_amcl` needs: a simulated 36-beam laser scan (ray-cast on the map) and the odometry transform. When it detects a kidnap, it asks AMCL to relocalize globally.
5. **Nav2 AMCL** – the existing MCL package used for comparison (Section 5).
6. **RViz, live plots and results logger** – visualize the robot, landmarks and both particle clouds, plot the error of both filters, and save the errors to CSV files.

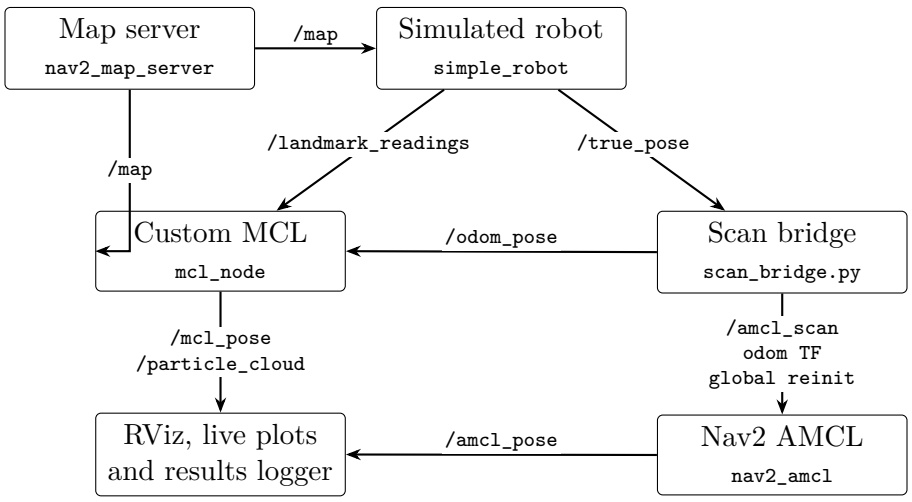


Figure 1: ROS 2 node communication graph. `/map` also goes to the scan bridge, AMCL and RViz, and `/true_pose` also goes to the plots and the results logger.

3.2 Test Maps

Figure 2 shows the maps used in the experiments. The five landmarks are at the same positions on every map; they are treated as point beacons, so they may lie inside obstacles. Map 1 is symmetric under a 180° rotation and Map 2 under 90° rotations. Map 4 is Map 1 with two extra obstacles that remove the symmetry. The symmetry does not matter for

our filter, which knows which landmark each reading belongs to, but it matters for AMCL, which only sees a laser scan (Section 5.5).

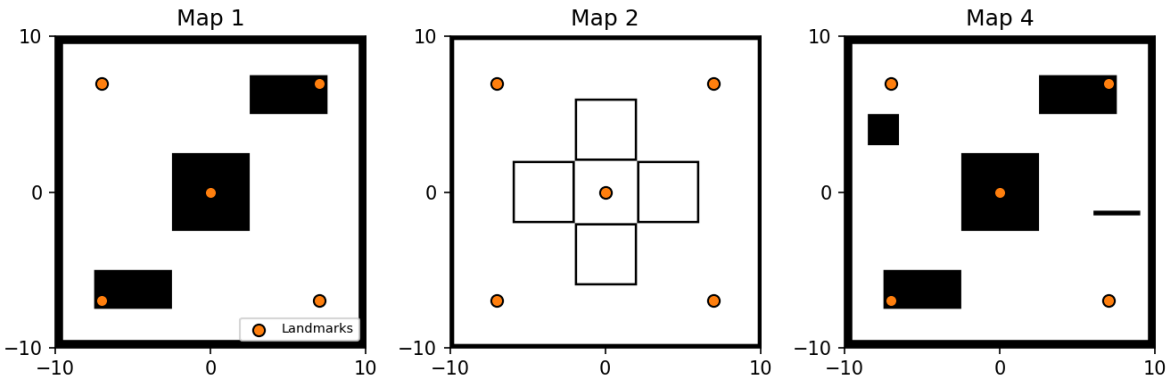


Figure 2: The maps used in the experiments, with the landmark positions. Each map is 20 m × 20 m.

4 Results

4.1 RViz Visualization

The following screenshots from RViz show the custom MCL running on Map 1 and Map 2 after it has converged. The particles are shown as red dots: most of them form a tight cluster at the robot (the cyan marker), and the scattered dots are the 15% random particles injected at every resampling step. The blue arrow is the MCL pose estimate.

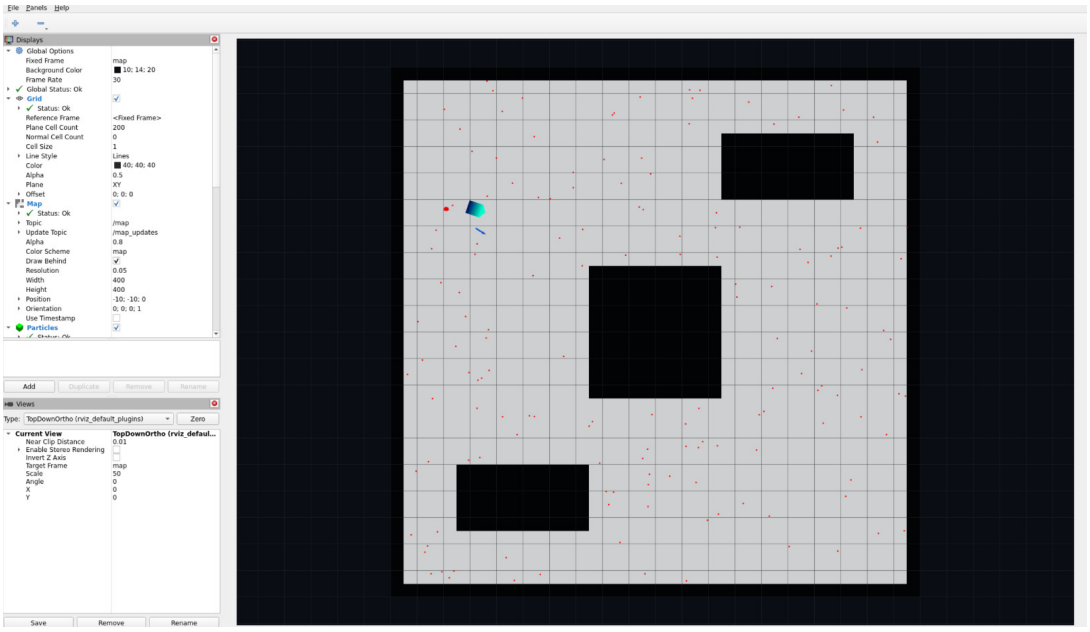


Figure 3: RViz visualization for Map 1

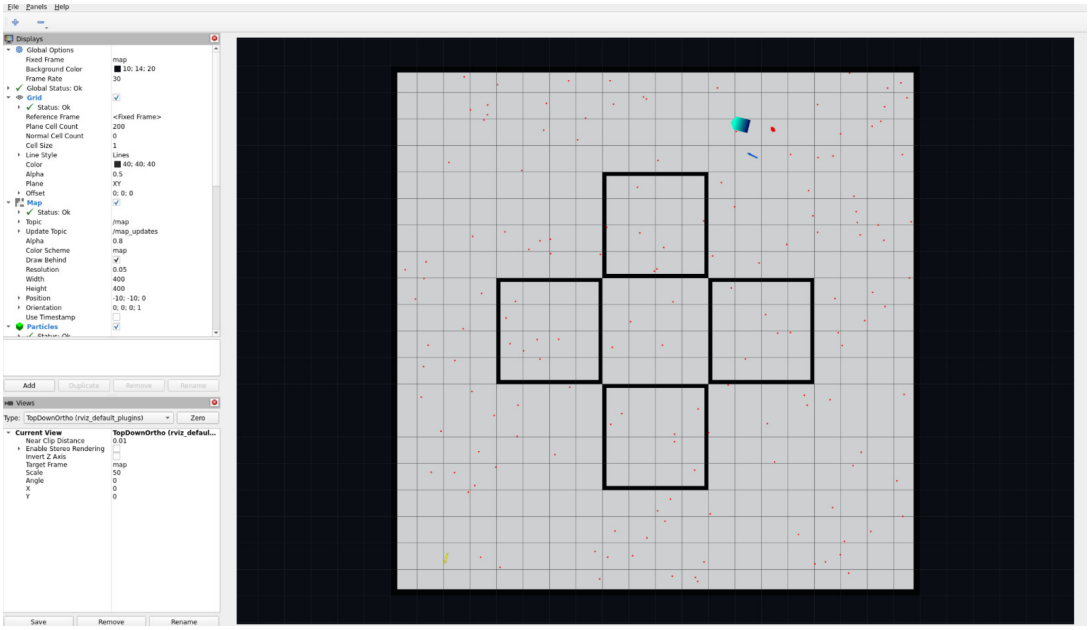


Figure 4: RViz visualization for Map 2

4.2 Localization Error Over Time

The localization error at time t is defined as the Euclidean distance between the estimated pose and the ground-truth pose:

$$e_t = \sqrt{(\hat{x}_t - x_t^*)^2 + (\hat{y}_t - y_t^*)^2}, \tag{21}$$

where (x_t^*, y_t^*) is the ground-truth position.

Figure 5 shows the error of the custom MCL over a 330 s run on Map 1 and on Map 2. At the start the particles are spread over the whole map and the error is about 8–9 m. In this run the filter converged to within 1 m in 6.1 s on Map 1 and 6.6 s on Map 2.

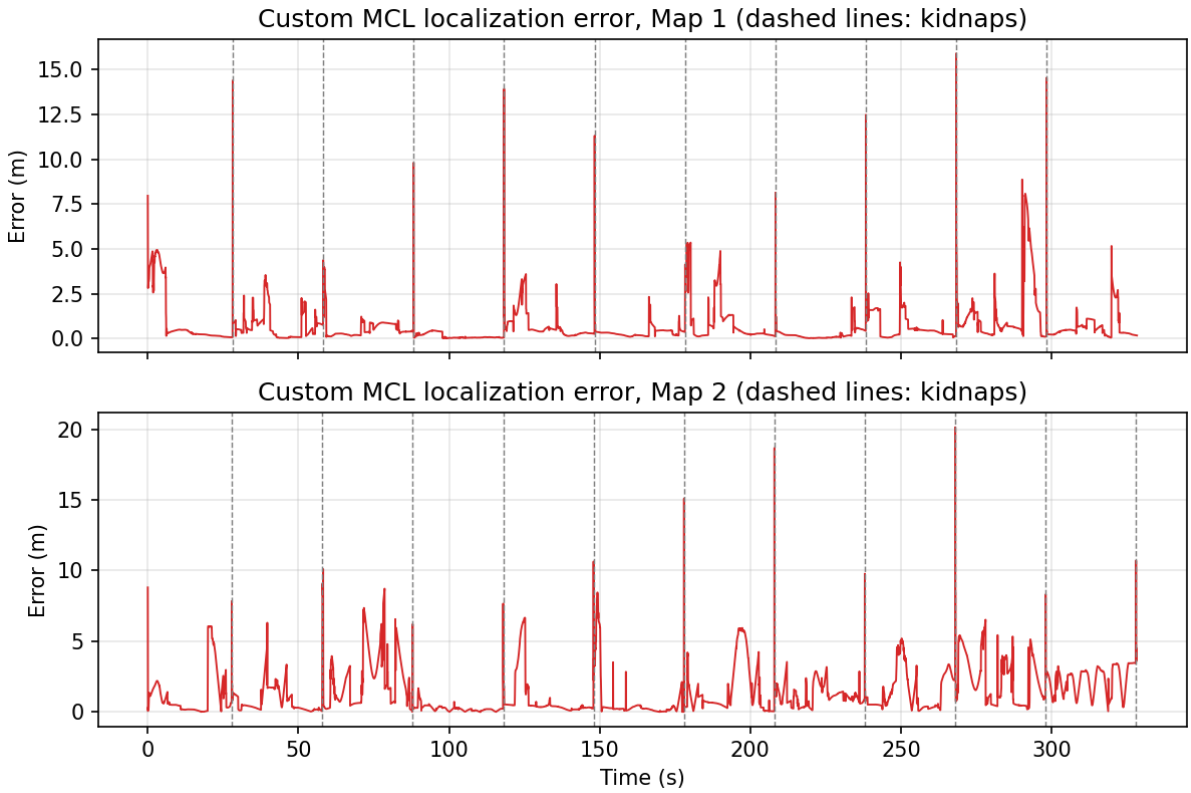


Figure 5: Localization error of the custom MCL over time on Map 1 and Map 2. The dashed lines mark the kidnaps.

The large spikes at the dashed lines happen every 30 s, when the robot is teleported to a random position on the map (the kidnapped robot problem). At that moment the particles are still at the old position, so the error jumps to the distance of the teleport. The particles that are injected at random then land near the new position, receive high weights, and the cloud re-forms there. The smaller spikes between kidnaps happen in parts of the map where only one or two landmarks are within the 8 m sensor range, for example near the corners: with a single landmark the range and bearing only constrain the robot to a circle around it, so the estimate can drift until more landmarks come into view.

We measure the recovery after each kidnap as the time until the error stays below 1 m for 3 s, and count a kidnap as recovered if the median error over the last 10 s before the next kidnap is below 1 m. Over two such runs per map (20 kidnaps each), the filter recovered from 17 of 20 kidnaps on Map 1 (median recovery time 2.8 s) and from 15 of 20 on Map 2 (median 2.3 s). After recovering, the median error was 0.29 m on Map 1 and 0.40 m on Map 2. The larger error on Map 2 comes from periods in which the estimate is off by 2–5 m for several seconds, for example after 240 s in Figure 5.

4.3 Effect of the Number of Particles

To see how the number of particles affects the filter, we ran it on Map 4 with $N \in \{50, 100, 250, 500, 1000\}$, twice for 230 s each, which gives 12 complete kidnaps per value. Figure 6 shows the share of kidnaps the filter recovered from and the error after recovery.

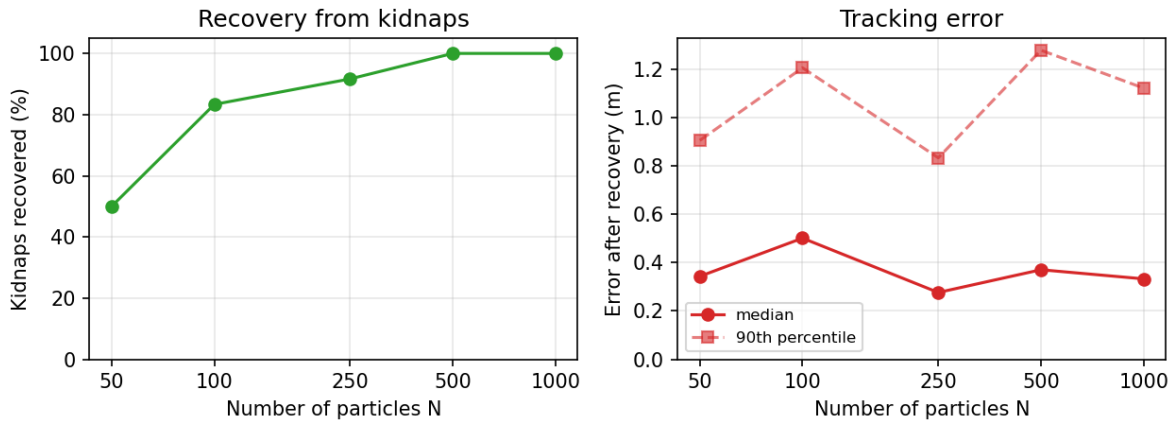


Figure 6: Effect of the number of particles on Map 4, over 12 kidnaps per value of N. Left: share of kidnaps after which the filter re-localized. Right: error after recovery.

The number of particles mainly affects **whether** the filter recovers from a kidnap. With $N = 50$ it recovered from only 6 of 12 kidnaps, with 100 from 10 of 12 and with 250 from 11 of 12; with 500 and 1000 it recovered every time. After a kidnap, the filter can only re-localize if some of the randomly injected particles (15% of N, so only 7 when $N = 50$) land close to the new position. With few particles this often does not happen before the existing cloud has locked onto a wrong position.

Once the filter had recovered, the tracking error did not depend on N in any consistent way: the median stayed between 0.28 and 0.50 m for all values, and the recovery time between 1 and 4 s. The accuracy is limited by the measurements rather than by the number of samples: with only a few landmarks within 8 m, and range noise of 0.2 m, more particles do not give a better estimate of the pose. The Python node kept up with the 20 Hz odometry rate for all values of N. We therefore use $N = 500$, the smallest value that recovered from every kidnap in this test.

5 Comparison with Existing MCL Package

We compare our from-scratch implementation against the widely-used `nav2_amcl` (Adaptive Monte Carlo Localization) package from the ROS 2 Navigation2 stack.

5.1 Overview of `nav2_amcl`

The `nav2_amcl` package implements an adaptive variant of MCL that uses:

- **KLD-sampling** to dynamically adjust the number of particles based on the complexity of the posterior distribution.
- **Likelihood field** or **beam** sensor models for laser range finders instead of landmark-based measurements.
- **Augmented MCL** to handle the kidnapped robot problem by injecting random particles when the average observation likelihood is low.
- An occupancy-grid map instead of a landmark-based map.
- nav2_amcl GitHub Repository link

5.2 Feature Comparison

Table 1: Feature comparison between our implementation and `nav2_amcl`.

Feature	**Our Implementation**	**nav2_amcl**
Motion model	Odometry (rot1-trans-rot2)	Odometry (differential drive)
Sensor model	Range-bearing to known landmarks	Likelihood field (36-beam laser)
Map	Landmark positions (+ grid for free space)	Occupancy grid
Particle count	Fixed, $N = 500$	KLD-sampling, 500–20 000
Resampling	Low-variance, every update	Low-variance, every update
Kidnapped robot	15% random injection + lost detection	Random injection + global re-initialisation
Implementation	Python	C++

5.3 Experimental Setup

Both filters run at the same time on the same simulated robot and map, and receive the same odometry. The custom MCL uses the landmark readings; AMCL uses a 36-beam laser scan that the scan bridge ray-casts on the map from the robot's true pose (range 15 m). AMCL is started with a global (uniform) initialisation. When the scan bridge sees the true pose jump, it calls AMCL's `reinitialize_global_localization` service, which spreads the particles over the whole map again. This gives AMCL a hint that our filter does not get: the custom MCL is not told about kidnaps and has to detect them through its own weights.

AMCL needs many particles to relocalize from a uniform spread over a 20 m × 20 m map. With at most 1000 particles, it almost never converged again after a kidnap (its error stayed at 7–9 m), so we allow up to 20 000; KLD-sampling reduces the count towards the minimum of 500 once the filter has converged.

We ran each of Map 1, Map 2 and Map 4 twice for 330 s, which gives 20 kidnaps per map, and recorded the true pose and both estimates. Recovery time and success are measured as in Section 4. In addition, when AMCL did not recover, we checked whether its estimate was within 0.5 m of a rotated copy of the true pose (180° for Map 1, 90°, 180° or 270° for Map 2).

5.4 Results

Table 2: Kidnap recovery over 20 kidnaps per map (two 330 s runs). Recovery time is the median over the recovered kidnaps. Tracking error is the error after recovery, given as the median and the 10th–90th percentile range. The one AMCL kidnap on Map 2 that is in neither row ended at a pose that was neither correct nor a rotated copy.

		Map 1	**Map 2**	**Map 4**
Custom MCL	Recovered	17 / 20	15 / 20	19 / 20
	Recovery time	2.8 s	2.3 s	0.7 s
	Tracking error	0.29 m (0.05–0.96)	0.40 m (0.08–2.01)	0.25 m (0.06–1.19)
Nav2 AMCL	Recovered	11 / 20	5 / 20	19 / 20
	Converged to a rotated twin	9 / 20	14 / 20	1 / 20
	Recovery time	7.3 s	16.3 s	5.4 s
	Tracking error	0.06 m (0.02–0.13)	0.06 m (0.03–0.13)	0.06 m (0.02–0.14)

Figure 7 shows both filters on Map 4, which has no symmetry. Here AMCL recovered from 19 of 20 kidnaps (median 5.4 s) and then tracked the robot to within about 6 cm. In the remaining kidnap it settled on a rotated copy of the true pose, probably because the two extra obstacles are not visible from everywhere (the central block can hide them), so from some places the map still looks symmetric to the laser. Our filter recovered faster (median 0.7 s) because the landmark readings identify the robot's position directly, but it tracked less precisely, with a median error of 0.25 m. AMCL compares 36 laser beams against the whole map, while our filter only has the few landmarks that are within 8 m. At the start of the six runs, our filter converged from the uniform spread in 3.4–6.6 s and AMCL in 4.2–11.3 s.

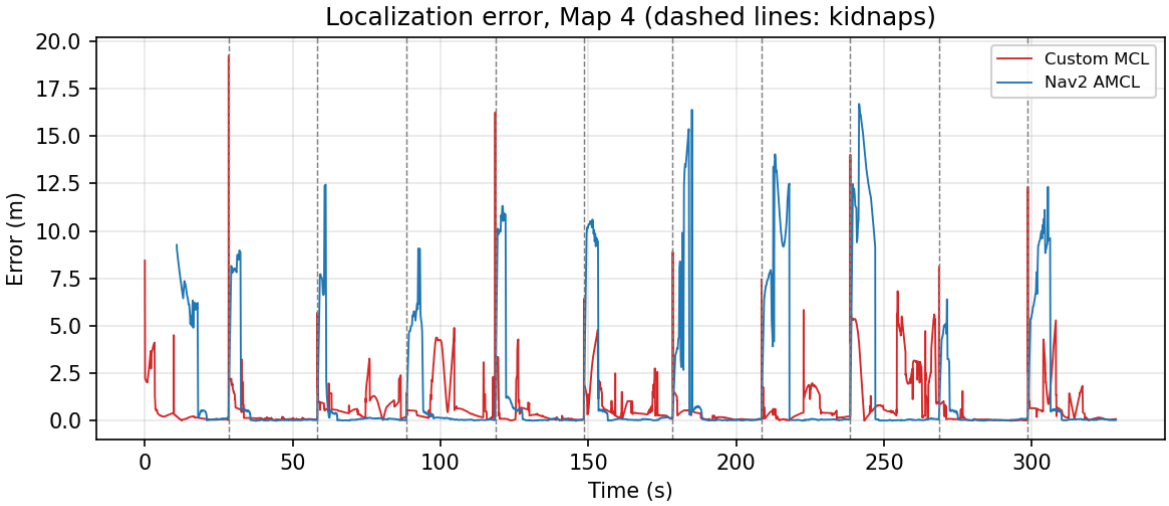


Figure 7: Localization error of both filters on Map 4.

5.5 Effect of Map Symmetry

On the symmetric maps, AMCL often settled on the wrong pose after a kidnap: on Map 1 in 9 of 20 kidnaps and on Map 2 in 14 of 20. In all of these cases its estimate was within 0.5 m of a rotated copy of the true pose; only once (on Map 2) did it end up somewhere else. On Map 1, a laser scan taken at (x, y, θ) is identical to one taken at $(-x, -y, \theta + \pi)$, so the two poses explain the measurements equally well and the filter has no way of choosing between them. Figure 8 shows this: whenever AMCL's error is large, its distance to the mirrored pose $(-x, -y)$ is close to zero. AMCL then either tracks the mirrored pose accurately until the next kidnap or switches back and forth between the two poses (the vertical jumps in the figure). Its random particle injection does not resolve the ambiguity, because the wrong pose fits the scans as well as the right one.

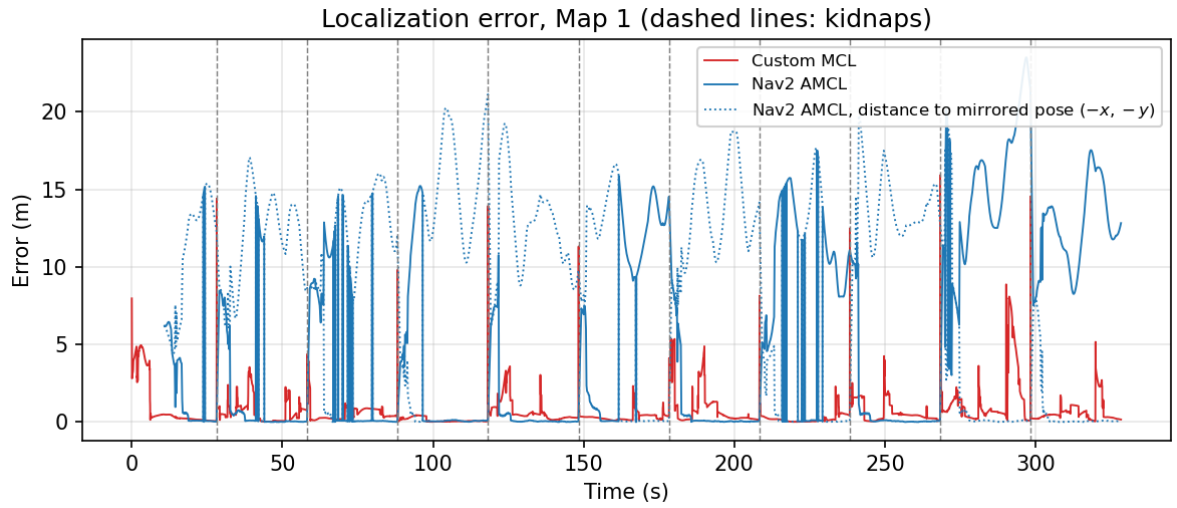


Figure 8: Localization error of both filters on Map 1. The dotted line is the distance from AMCL's estimate to the mirrored pose $(-x, -y)$ of the robot.

Our filter does not have this problem, even though the landmarks themselves are placed symmetrically, because every reading carries the index of the landmark it belongs to. This known data association is what removes the ambiguity, and it is the main advantage of the landmark-based approach in this comparison. With a laser alone, the map itself has to be asymmetric, as in Map 4.

6 Conclusion

6.1 Summary

In this project, we implemented a particle filter for Monte Carlo Localization entirely from scratch. The three core steps—predict (motion model with Gaussian noise), update (range-bearing measurement likelihood), and resample (low-variance resampling)—were implemented as a ROS 2 Python node and validated in a 2D grid world with known landmarks.

6.2 Key Findings

- **Global localization:** Starting with particles spread over the whole map (initial error about 8–9 m), the filter converged to within 1 m in 3.4–6.6 s.
- **Kidnapped robot recovery:** With 15% random particle injection and lost detection, the filter recovered from 51 of 60 kidnaps over the three maps, with a median recovery time of 0.7–2.8 s depending on the map, without being told when a kidnap happened.
- **Accuracy:** After recovering, the median error was 0.25–0.40 m. The error is larger where only one or two landmarks are within sensor range.
- **Comparison with `nav2_amcl`:** On an asymmetric map, AMCL recovered from 19 of 20 kidnaps and tracked the robot about four times more precisely (about 6 cm

against 0.25 m), but took longer to recover (median 5.4 s against 0.7 s) and needed up to 20 000 particles to relocalize over the whole map. On the symmetric maps it settled on a rotated copy of the true pose after 9 of 20 (Map 1) and 14 of 20 (Map 2) kidnaps, because a laser scan cannot tell those poses apart. Our filter is not affected by the symmetry because it knows which landmark each reading comes from.

- **Number of particles:** More particles made the recovery from kidnaps more reliable (from 6 of 12 kidnaps with $N = 50$ to 12 of 12 with $N \geq 500$), but did not reduce the tracking error, which is limited by the landmark measurements (Section 4.3).